

C Purchase Orders

From Sage Evolution SDK | Documentation Portal

The SDK only Allows for Supplier Invoice documents to be processed through PurchaseOrder class. Purchase Orders can either be placed by using the Save() method or they can be processed in to a Supplier invoice using the Process() method. When a order is placed it is just a request for goods and is not posted to any posting tables but gets saved as a document in the invnum table and affects the items quantities ordered. Once the order is processed the necessary posting tables are populated affecting the General Ledger, Supplier Ledger and Inventory. Since the ordered quantity might not be the received quantities orders can be partially processed or goods can be received while the actual supplier invoice remains unprocessed.

The following example display how a typical order is placed using the SDK then the order is partially processed then the balance of the order is completed:

```
//Create a new instance of a PurchaseOrder class
PurchaseOrder order = new PurchaseOrder();
Supplier supp = new Supplier("Supplier1");
order.Supplier = supp; //Assign a value to the Supplier property
//the Detail property contains the document lines
order.Detail.Add("ItemA", 5, 10);
order.Save(); //Place the Order
string orderNo = order.OrderNo;
MessageBox.Show(orderNo);
```

There are a number of overloaded **Add** methods. The **Add** method used above specifies the product code as a string, but you may well find that the other versions of this method will better suit your purpose. Assuming the account is taxable, the document total as well as the total for line 1 (index 0 in the collection) will result in 57.00, local currency. Every sales order must have an order number, so at this point, unless you are supplying your own order number, you may want to store the value of **order.OrderNo**, which is generated.

To process the Order in to a Supplier invoice we would need to specify a quantity to process and use the process method as follows.

```
order.Detail[0].ToProcess = 2;
order.Process();
```

If we set the **ToProcess** quantity to 5, we would in fact be completing the order when we call Process. The Process function takes care of all the underlying transactions and returns the reference number of the invoice transaction created. Thereafter there would be no point in keeping this order opened, since the quantity of stock requested has now been fully received; hence the order is then marked as archived and cannot be processed further. By setting the ToProcess quantity above to 2, you would end up with 2 documents: an archived document for 2 units, and a partially processed document for the remaining 3 units. The archived document cannot be altered, but we can later retrieve the partially processed document. By using the complete method we can even process the total quantity outstanding automatically and not specifying a quantity as below

```
PurchaseOrder newOrder = new PurchaseOrder(orderNo);
newOrder.Complete(); // the complete method completes the whole unprocessed order
```

The following example displays a Purchase order using a GL line as well as a inventory item also we specify addresses to be used. To use a GL Account as a item the Account in Evolution has to first be edited and flagged to be used on a order:

```
PurchaseOrder PO = new PurchaseOrder();
PO.Supplier = new Supplier("SupplierSDK1");
PO.InvoiceDate = DateTime.Now; // choose to set the invoice date or Order date etc
PO.InvoiceTo = PO.Supplier.PostalAddress.Condense(); // Condense method can be used or you can specify the address
PO.DeliverTo = new Address("Physical Address 1", "Address 2", "Address 3", "Address 4", "Address 5", "PC");
PO.Project = new Project("P1"); // Various PO properties like project can be set

OrderDetail OD = new OrderDetail();
PO.Detail.Add(OD);
// Various Order Detail properties can be added like warehouse, sales reps, userfields etc
OD.InventoryItem = new InventoryItem("ItemA"); // Use the inventoryItem constructor to specify a Item
OD.Quantity = 10;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = 20;

OD = new OrderDetail();
PO.Detail.Add(OD);
OD.GLAccount = new GLAccount("Accounting Fees"); // Use the GLAccount Item constructor to specify a Account
OD.Quantity = 1;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = 30;

PO.Process();
```

You can also receive Stock using the SDK and process the Supplier invoice later. To do this make sure Evolution | Order Entry | Defaults | Purchase Orders is set to Split GRV from SINV. Also make sure the Account types on the Inventory SINV transaction type is set up correctly.

```
PurchaseOrder PO = new PurchaseOrder();
PO.Supplier = new Supplier("SupplierSDK1");

OrderDetail OD = new OrderDetail();
PO.Detail.Add(OD);
OD.InventoryItem = new InventoryItem("ItemWA");
OD.Warehouse = new Warehouse("mstr");
OD.Quantity = 10;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = 20;

PO.ProcessStock(); // Process the GRV and receive stock while supplier invoice is unprocessed

//////////Process the unprocessed Supplier Invoice//////////

PurchaseOrder SINV = new PurchaseOrder("P00023", "GRV0028"); // Specify the order number and GRV to get the unprocessed
foreach (OrderDetail NewOD in SINV.Detail)
{
    NewOD.ToProcess = NewOD.Outstanding; // specify outstanding quantity to process or change price if needed
}
SINV.Process(); // Process the SINV
```

The following example displays how to specify Additional Costs on a Purchase Order:

```
PurchaseOrder PO = new PurchaseOrder();
PO.Supplier = new Supplier("SupplierSDK1");

// Additional costs can be specified on a Purchase Order Invoice like the functionality in Evolution
CostAllocation costAlloc = new CostAllocation();
costAlloc.Supplier = new Supplier("Supplier1");
costAlloc.Reference = "ADDCost";
costAlloc.Description = "AddCost";
costAlloc.Amount = 200;
costAlloc.TaxRateID = 3;
costAlloc.Save(); // Save the additional cost lines

PO.AdditionalCosts.Add(costAlloc); // Add the total additional costs to the PO

OrderDetail OD = new OrderDetail();
PO.Detail.Add(OD);
OD.InventoryItem = new InventoryItem("ItemA");
OD.TotalAdditionalCost = 100; // Specify the Additional Costs per line or remove and use the distribute method
OD.Quantity = 1;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = 20;
```

```
OD = new OrderDetail();
PO.Detail.Add(OD);
OD.InventoryItem = new InventoryItem("ItemB");
OD.TotalAdditionalCost = 100;
OD.Quantity = 1;
OD.ToProcess = OD.Quantity;
OD.UnitSellingPrice = 30;

// PO.AdditionalCosts._Distribute();// Use this method when not specifying additional costs per line and to dist
PO.Process();
```

Inventory documents can be populated in tax exclusive or tax inclusive mode using the TaxMode property on the OrderBase record. When creating a new Order record or loose-standing OrderDetail object, TaxMode will default to TaxExclusive. However, when using one of the OrderDetailCollection Add methods, the detail object will default to whatever the current tax mode on the document is. Set the tax mode before setting the unit price.

```
PurchaseOrder pord = new PurchaseOrder();
pord.Account = new Supplier("ABC009");
pord.TaxMode = TaxMode.Inclusive;
```

How to specify my own invoice number?

You will notice the inventory document classes contain multiple overloads of the Complete and Process methods. Those methods accepting a 'reference' parameter will use that reference as the document invoice number. If one of the other methods is used, or a blank reference parameter supplied, an invoice number will be generated. If number generation is disabled in inventory defaults, an exception will follow. The final reference number used - whether generated or supplied - will become the function's return value.

Retrieved from "https://developerzone.pastel.co.za/index.php?title=C_Purchase_Orders&oldid=169"

-
- This page was last modified on 26 October 2015, at 11:26.